HDOC Day-72

Student Attendance Management Using Dynamic Memory Allocation and Pointers in C

Aim

To develop a C program that manages student attendance using a dynamically resizable matrix, pointer notation, separate user-defined functions, malloc(), realloc(), and free(). Attendance is represented by 1 (Present) or 0 (Absent).

Algorithm

1. Read and validate the initial number of students and working days.

2. Allocate a contiguous attendance matrix dynamically using malloc() and check for allocation failure.

3. Record attendance for every student and day, accepting only 0 or 1.

4. Display the initial attendance matrix.

5. Read the number of additional students and validate it.

6. Use realloc() to enlarge the matrix when additional students are requested, retaining existing attendance values.

7. Record attendance for the newly added students.

8. Display the updated matrix and calculate each student's attendance percentage.

9. Release all dynamically allocated memory using free() before termination.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <limits.h>

int *createMatrix(int students, int days);
void recordAttendance(int *attendance, int start,
                      int students, int days);
int *addStudents(int *attendance, int *students,
                 int additional, int days);
void displayAttendance(const int *attendance,
                       int students, int days);
void calculateAttendance(const int *attendance,
                         int students, int days);
void releaseMemory(int *attendance);

int main(void)
{
    int students, days, additional;
    int *attendance;

    printf("Enter initial number of students: ");
    if (scanf("%d", &students) != 1 || students <= 0)
    {
        printf("Invalid number of students!\n");
        return 1;
    }

    printf("Enter number of working days: ");
    if (scanf("%d", &days) != 1 || days <= 0)
    {
```

```
        printf("Invalid number of working days!\n");
        return 1;
    }

    attendance = createMatrix(students, days);
    if (attendance == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("\nEnter initial attendance details:\n");
    recordAttendance(attendance, 0, students, days);

    printf("\nInitial Attendance Matrix:\n");
    displayAttendance(attendance, students, days);

    printf("\nEnter number of additional students: ");
    if (scanf("%d", &additional) != 1 || additional < 0)
    {
        printf("Invalid number of additional students!\n");
        releaseMemory(attendance);
        return 1;
    }

    if (additional > 0)
    {
        int *temp = addStudents(attendance, &students,
                                additional, days);
        if (temp == NULL)
        {
            printf("Memory reallocation failed!\n");
            releaseMemory(attendance);
            return 1;
        }
        attendance = temp;
    }

    printf("\nUpdated Attendance Matrix:\n");
    displayAttendance(attendance, students, days);

    printf("\nAttendance Calculation:\n");
    calculateAttendance(attendance, students, days);
    releaseMemory(attendance);
    printf("\nMemory released successfully.\n");
    return 0;
}

int *createMatrix(int students, int days)
{
    size_t total;
    if ((size_t)students > SIZE_MAX / (size_t)days)
        return NULL;
    total = (size_t)students * (size_t)days;
    if (total > SIZE_MAX / sizeof(int))
        return NULL;
    return malloc(total * sizeof(int));
}

void recordAttendance(int *attendance, int start,
                      int students, int days)
{
    int i, j, value;
    for (i = start; i < students; i++)
    {
        printf("\nStudent %d:\n", i + 1);
        for (j = 0; j < days; j++)
        {
            while (1)
            {
                printf("Day %d (1=Present, 0=Absent): ",
```

```
                       j + 1);
                if (scanf("%d", &value) != 1)
                {
                    int ch;
                    while ((ch = getchar()) != '\n' &&
                           ch != EOF) { }
                    printf("Invalid input! Enter 0 or 1.\n");
                    continue;
                }
                if (value == 0 || value == 1)
                {
                    *(attendance + i * days + j) = value;
                    break;
                }
                printf("Invalid attendance! Enter 0 or 1.\n");
            }
        }
    }
}

int *addStudents(int *attendance, int *students,
                 int additional, int days)
{
    int oldStudents = *students;
    int newStudents;
    int *temp;
    size_t total;
    if (additional <= 0 || additional > INT_MAX - oldStudents)
        return NULL;
    newStudents = oldStudents + additional;
    if ((size_t)newStudents > SIZE_MAX / (size_t)days)
        return NULL;
    total = (size_t)newStudents * (size_t)days;
    if (total > SIZE_MAX / sizeof(int))
        return NULL;
    temp = realloc(attendance, total * sizeof(int));
    if (temp == NULL)
        return NULL;
    *students = newStudents;
    printf("\nEnter attendance for new students:\n");
    recordAttendance(temp, oldStudents, newStudents, days);
    return temp;
}

void displayAttendance(const int *attendance,
                       int students, int days)
{
    int i, j;
    printf("Student\t");
    for (j = 0; j < days; j++)
        printf("Day%d\t", j + 1);
    printf("\n");
    for (i = 0; i < students; i++)
    {
        printf("%d\t", i + 1);
        for (j = 0; j < days; j++)
            printf("%d\t", *(attendance + i * days + j));
        printf("\n");
    }
}

void calculateAttendance(const int *attendance,
                         int students, int days)
{
    int i, j, present;
    double percentage;
    for (i = 0; i < students; i++)
    {
        present = 0;
        for (j = 0; j < days; j++)
```

```
            if (*(attendance + i * days + j) == 1)
                present++;
        percentage = (double)present / days * 100.0;
        printf("Student-%d: %.1f%%\n", i + 1, percentage);
    }
}

void releaseMemory(int *attendance)
{
    free(attendance);
}
```

Explanation

Dynamic allocation: malloc() allocates students x days integer slots as one contiguous block.

Pointer notation: The expression *(attendance + i * days + j) accesses the attendance of student i on day j.

Resizing: realloc() enlarges the allocation to hold the added student rows while preserving existing records.

Attendance calculation: Attendance percentage = (days present / total working days) x 100.

Memory management: free() releases the allocated matrix before the program ends.

Worked Example

Initially, there are 3 students and 3 working days. Three additional students are then added.

Initial attendance matrix

Student	Day 1	Day 2	Day 3
Student 1	1	0	0
Student 2	1	0	1
Student 3	1	1	1

Updated attendance matrix

Student	Day 1	Day 2	Day 3
Student 1	1	0	0
Student 2	1	0	1
Student 3	1	1	1
Student 4	1	0	0
Student 5	1	1	1
Student 6	1	0	1

Attendance percentages: Student 1: 33.3%, Student 2: 66.7%, Student 3: 100.0%, Student 4: 33.3%, Student 5: 100.0%, Student 6: 66.7%.

Test Case Table

Case	Input / scenario	Expected result
1	3 students, 3 days	Initial matrix created
2	Add 3 students	Matrix grows to 6 students
3	Present all days	100% attendance
4	Absent all days	0% attendance
5	Present 2 of 3 days	66.7% attendance
6	Attendance value 2	Prompt again
7	Add 0 students	No resizing
8	Initial students = 0	Invalid input
9	Working days = -2	Invalid input
10	Allocation failure	Error message

Sample Output

```
Enter initial number of students: 3
Enter number of working days: 3

Enter initial attendance details:
Student 1: 1 0 0
Student 2: 1 0 1
Student 3: 1 1 1

Initial Attendance Matrix:
Student  Day1  Day2  Day3
1        1     0     0
2        1     0     1
3        1     1     1

Enter number of additional students: 3
Enter attendance for new students:
Student 4: 1 0 0
Student 5: 1 1 1
Student 6: 1 0 1

Updated Attendance Matrix:
Student  Day1  Day2  Day3
1        1     0     0
2        1     0     1
3        1     1     1
4        1     0     0
5        1     1     1
6        1     0     1

Attendance Calculation:
Student-1: 33.3%
Student-2: 66.7%
Student-3: 100.0%
Student-4: 33.3%
Student-5: 100.0%
Student-6: 66.7%

Memory released successfully.
```

Invalid attendance input: Entering 2 prompts "Invalid attendance! Enter 0 or 1." and requests the value again.

Invalid dimensions: Entering 0 initial students displays "Invalid number of students!".

Result

The C program manages student attendance using a dynamically allocated and resizable matrix. It records and validates attendance, adds students using realloc(), preserves previous records, calculates attendance percentages, and releases memory using free(). Separate user-defined functions and pointer arithmetic are used as required.